Packages

What is a package?

A **package** is a way to **organize Python code in folders**. It groups related modules (Python files) together so your project stays clean and scalable.

- **Module** = one **.py** file (example: math_utils.py)
- **Package** = a **folder** containing **multiple modules** (and possibly subfolders/sub-packages)
- **Library** = a **collection** of **modules/packages** you usually install via **pip** (example: requests, numpy)

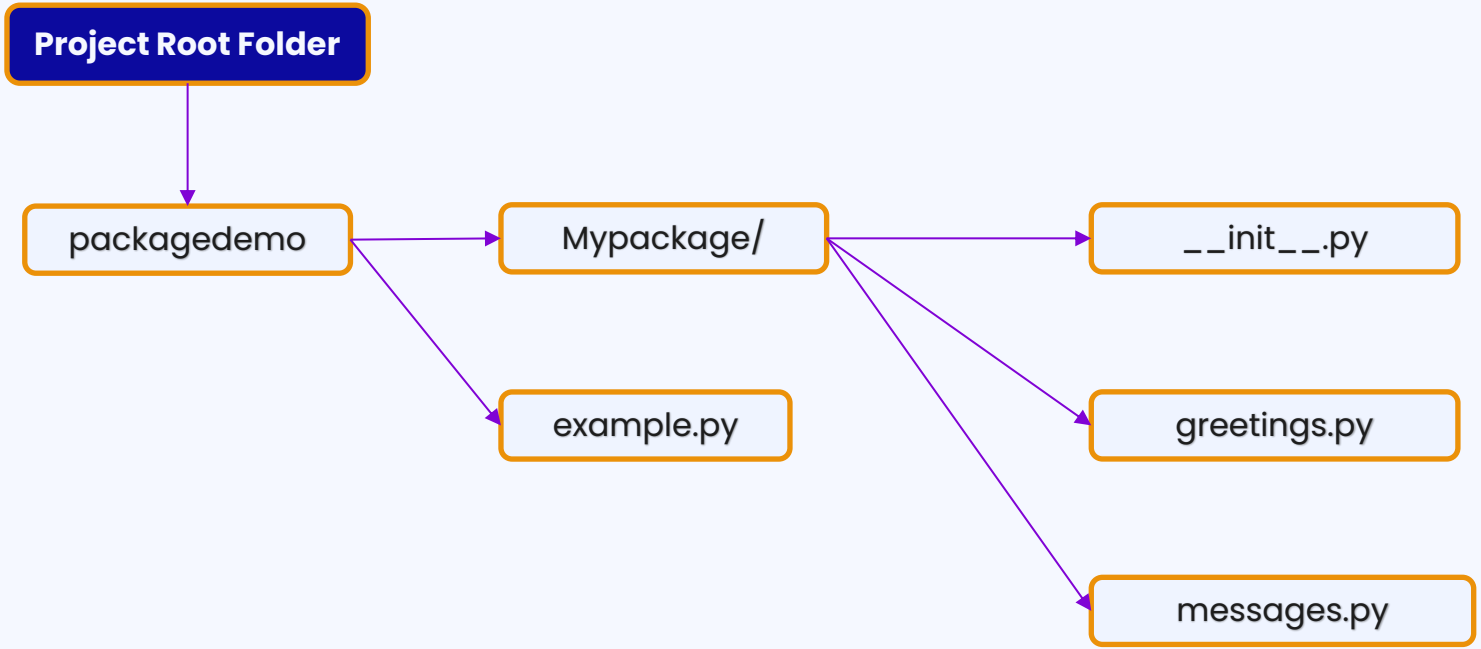
Why packages are used

As **projects** grow, keeping everything in **one file** becomes **messy**.

Packages help you structure code like **“features”**.

- **Better organization** (auth, db, services, utils)
- **Easy reuse of code**
- **Cleaner imports**
- **Team projects** become manageable

Structure


Project Root Folder
packagedemo
Mypackage/
example.py
__init__.py
greetings.py
messages.py

What is __init__.py?

__init__.py is a file that marks a directory as a package (historically required, still commonly used).

What it can do:

- **Make** imports easier
- **Expose** selected functions/classes (public API)
- **Run** initialization code (careful with side effects)
- **Control** what gets imported using *

What is __init__.py?

Example idea:

Python

```python
# mypackage/__init__.py
from .calc import add, sub
```

Now you can do:

Python

```python
from mypackage import add
```

Modern Python can treat folders as **“namespace packages”** even without __init__.py, but for course-level understanding and real projects, **using __init__.py is best practice.**

Writing the Package Modules

Module 1: greetings.py

Python

```python
def say_hello (name):
        return f"Hello, {name}!“

def say_goodbye (name):
        return f"Goodbye, {name}!“
```

Module 2: messages.py

Python

```python
def welcome():
        return "Welcome to our application! “

def thank_you():
        return "Thank you for using our service!"
```

Using the Package (Importing Modules)

Now we will test the package using a script placed **above the package folder**.

Python (example.py)	Output

`from mypackage.greetings import say_hello`
`print(say_hello("Aman"))`
 	
`from mypackage.messages import welcome`
`print(welcome())`
	`Hello, Aman!`
	
	`Welcome to our application!`

Defining a Package List using __init__.py

Instead of importing from individual modules every time, we can **expose selected functions** using **__init__.py**.

Python (__init__.py)

```python
from .greetings import say_hello

from .messages import thank_you
```

Now these functions become **directly accessible from the package**.

Using the Package List

Create another test file above the package folder.

Python (testpackage.py)	Output

`from mypackage import say_hello, thank_you`
 	
`print(say_hello("Aman"))`
`print(thank_you())`
	`Hello, Neha!`
	
	`Thank you for using our service!`

This shows how **__init__.py** controls **what the package exports**.

Installing the Package using PIP

- The package works only when scripts are placed **near the package folder**.
- To use the package **anywhere**, we need to install it.

Step 1: Create setup.py

Python (setup.py)

```python
from setuptools import setup

setup(
    name='mypackage',
    version='0.1',
    description='Simple demo package',
    author='amar',
    packages=['mypackage']
)
```

Installing the Package using PIP

- The package works only when scripts are placed **near the package folder**.
- To use the package **anywhere**, we need to install it.

Step 2: Install the package

Open the command prompt in the **packagedemo** folder and run:

cmd

```
pip install .
```

After installation, Python **registers** the package.

Using the Installed Package Anywhere

- Now you can use the package from **any Python session**.

cmd

```
>>> import mypackage
>>> mypackage.say_hello("Ravi")
'Hello, Ravi!'
```

Practice Set – 1

Download Link in
Description
and
Pinned Comment

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Practice Set – 1 Solution